

Nome:

Nota:

QUADRO DE RESPOSTAS

Q1

A)

B)

C)

D)

E)

F)

G)

Q2

A)

B)

C)

D)

E)

Q3

Q4

Q5

Q6

CENÁRIO

Considere o sistema de uma locadora de carros representado pelos códigos abaixo. Analise as classes e suas relações para responder às questões.

Carro.java

```
1 public class Carro {
2     private String modelo;
3     private double valorDiaria;
4
5     public Carro(String modelo, double valorDiaria) {
6         this.modelo = modelo;
7
8         if (valorDiaria >= 0) {
9             this.valorDiaria = valorDiaria;
10        }
11    }
12
13    public double getValorDiaria() {
14        return valorDiaria;
15    }
16
17    public void alterarValorDiaria(double novoValor) {
18        if (novoValor >= 0) {
19            valorDiaria = novoValor;
20        }
21    }
22 }
```

Locacao.java

```
1 public class Locacao {
2     private Carro carro;
3     private int quantidadeDias;
4
5     public Locacao(Carro carro, int quantidadeDias) {
6         this.carro = carro;
7
8         if (quantidadeDias >= 0) {
9             this.quantidadeDias = quantidadeDias;
10        }
11    }
12
13    public Carro getCarro() {
14        return carro;
15    }
16
17    public int getQuantidadeDias() {
18        return quantidadeDias;
19    }
20
21    public void aumentarDias(int quantidade) {
22        if (quantidade > 0) {
23            quantidadeDias += quantidade;
24        }
25    }
26
27    public double calcularValor() {
28        return carro.getValorDiaria() * quantidadeDias;
29    }
30 }
```

Patio.java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Patio {
5     private List<Carro> carros;
6
7     public Patio() {
8         carros = new ArrayList<>();
9     }
10
11    public void adicionarCarro(Carro carro) {
12        carros.add(carro);
13    }
14
15    public int contarDiariasAcima(double limite) {
16        int quantidade = 0;
17
18        for (Carro carro : carros) {
19            if (carro.getValorDiaria() >= limite) {
20                quantidade = quantidade + 1;
21            }
22        }
23
24        return quantidade;
25    }
26 }
```

Fatura.java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Fatura {
5     private List<Locacao> locacoes;
6
7     public Fatura() {
8         locacoes = new ArrayList<>();
9     }
10
11    public void adicionarLocacao(Locacao locacao) {
12        locacoes.add(locacao);
13    }
14
15    public double calcularTotal() {
16        double total = 0.0;
17
18        for (Locacao locacao : locacoes) {
19            total += locacao.calcularValor();
20        }
21
22        return total;
23    }
24 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Carro compacto = new Carro("Argo", 120.0);
4         Locacao principal = new Locacao(compacto, 3);
5
6         System.out.println(principal.calcularValor());
7
8         compacto.alterarValorDiaria(140.0);
9         System.out.println(principal.calcularValor());
10
11        Locacao apoio = principal;
12        apoio.aumentarDias(1);
13        System.out.println(principal.calcularValor());
14
15        Locacao segunda = new Locacao(compacto, 2);
16        System.out.println(segunda.calcularValor());
17
18        Carro apoioCarro = compacto;
19        apoioCarro.alterarValorDiaria(150.0);
20
21        System.out.println(principal.calcularValor());
22        System.out.println(segunda.calcularValor());
23
24        apoio.aumentarDias(1);
25        System.out.println(principal.calcularValor());
26    }
27 }
```

QUESTÕES

Q1. (35 pontos) Considere a execução completa da classe **Main** e indique o valor exibido no console em cada impressão abaixo.

- A) Linha 6
- B) Linha 9
- C) Linha 13
- D) Linha 16
- E) Linha 21
- F) Linha 22
- G) Linha 25

Q2. (25 pontos) Analise as afirmativas abaixo e assinale (V) para as verdadeiras e (F) para as falsas.

- A) Ao executar `new Carro("Teste", -50.0)`, o campo `valorDiaria` permanece com `0.0`, pois o valor negativo não é incorporado ao estado pelo construtor.
- B) Depois que a classe `Carro` declara `Carro(String modelo, double valorDiaria)`, Java continua fornecendo automaticamente um construtor sem argumentos, portanto `new Carro()` também permanece válido.
- C) Em `new Carro("Argo", 120.0)`, "Argo" e 120.0 são argumentos fornecidos na criação; `modelo` e `valorDiaria`, declarados no construtor, são parâmetros que recebem esses valores.
- D) Na instrução `this.valorDiaria = valorDiaria`, `this.valorDiaria` identifica o campo do objeto atual, enquanto `valorDiaria`, sem `this`, identifica o parâmetro recebido pelo construtor.
- E) Se a regra que impede valores negativos ficar apenas em `Main`, qualquer outro código que use o construtor também estará automaticamente impedido de fornecer um valor negativo.

Q3. (10 pontos) Considere os objetos abaixo:

```
Carro primeiro = new Carro("Mobi", 100.0);
Carro segundo = new Carro("Mobi", 100.0);

Locacao locacaoA = new Locacao(primeiro, 2);
Locacao locacaoB = new Locacao(segundo, 2);
```

Qual alternativa descreve corretamente a identidade desses objetos?

- A) `primeiro == segundo` é `true`, pois os dois carros possuem o mesmo modelo e o mesmo valor de diária.
- B) `locacaoA == locacaoB` é `true`, pois as duas locações produzem o mesmo valor.
- C) `primeiro == segundo` e `locacaoA == locacaoB` são `false`; expressões `new` diferentes criaram objetos com identidades distintas, mesmo que seus estados coincidam.
- D) `locacaoA.getCarro() == locacaoB.getCarro()` é `true`, pois os dois carros possuem valores iguais em seus campos.
- E) Como os dois carros possuem o mesmo estado, Java mantém apenas um deles e faz as duas variáveis apontarem automaticamente para a mesma identidade.

Q4. (10 pontos) Considere agora uma classe **Fatura** que reúne várias locações e precisa calcular o valor total a pagar. Cada **Locacao** já possui o método **calcularValor()**.

Qual proposta distribui melhor as responsabilidades?

- A) **Fatura** deve percorrer suas locações, obter diretamente de cada uma o carro e a quantidade de dias e refazer internamente o cálculo do valor da locação.
- B) **Fatura** deve percorrer suas locações e solicitar `locacao.calcularValor()` a cada uma, acumulando os resultados.
- C) Cada **Carro** deve conhecer todas as locações em que aparece e calcular o valor total da fatura.
- D) **Main** deve manter a lista de locações e fazer toda a soma, pois é o código que cria os objetos.
- E) **Fatura** deve copiar para seus próprios campos o valor da diária e a quantidade de dias de cada locação antes de calcular o total.

Q5. (10 pontos) Considere a implementação de **Patio** apresentada no cenário e o seguinte trecho:

```
Carro popular = new Carro("Mobi", 100.0);
Carro sedan = new Carro("Virtus", 140.0);

Patio patio = new Patio();
patio.adicionarCarro(popular);
patio.adicionarCarro(sedan);
patio.adicionarCarro(popular);

popular.alterarValorDiaria(150.0);
```

Qual valor será devolvido por `patio.contarDiariasAcima(130.0)`?

- A) 0
- B) 1
- C) 2
- D) 3
- E) O código não compila porque o mesmo objeto **Carro** não pode ser adicionado duas vezes à lista.

Q6. (10 pontos) Na classe **Fatura** apresentada no cenário, um estudante propõe acrescentar:

```
public List<Locacao> getLocacoes() {
    return locacoes;
}
```

Considerando que **Fatura** deve controlar sua própria coleção por meio de suas operações, qual análise é a mais adequada?

- A) A mudança enfraquece esse controle, pois quem receber a referência da lista poderá modificá-la diretamente sem passar pelas operações definidas por **Fatura**.
- B) A mudança é segura porque `locacoes` continua declarada como `private`; nenhuma alteração poderá ser feita fora da classe.
- C) A mudança cria automaticamente outra lista contendo cópias das locações, portanto a coleção interna continua isolada.
- D) A mudança é necessária, pois um campo `private` não pode ser percorrido pelos métodos da própria classe.
- E) A mudança transforma cada **Locacao** armazenada na lista em um objeto `public`, mas não permite modificar a coleção.